The open-source arsenal for building a browser-native agent OS

BlackRoad OS needs to orchestrate 30,000 concurrent AI agents in the browser while feeling like a real operating system. That's not a web app — it's a systems engineering challenge disguised as one. The good news: the open-source ecosystem in 2025/2026 has produced an extraordinary set of building blocks across agent infra, WASM runtimes, OS primitives, and dev tooling that map almost perfectly to this architecture. Below are the ~40 most compelling repositories across all four layers, mixing blockbusters with genuinely surprising hidden gems, each evaluated for what specifically could be forked, adapted, or studied.

Layer 1: Agent orchestration and AI infrastructure

This layer is the brain — how agents think, communicate, remember, and coordinate. The critical insight: **no single framework solves the full problem**, but several can be composed into a stack that handles LLM routing, agent memory, event-driven coordination, and capability management.

TensorZero — the Rust LLM gateway that changes the math

tensorzero/tensorzero · ~10,500 ⭐ · Rust, Python

A high-performance LLM gateway delivering **sub-1ms P99 latency at 10,000+ QPS** TensorZero Docs — roughly 25–100x faster than Python alternatives like LiteLLM under heavy load. VentureBeat It unifies model routing, observability, evaluation, and A/B testing into one binary GitHub with structured schema enforcement for all inputs and outputs. TensorZero Docs

For 30K agents hammering LLM APIs simultaneously, the gateway layer is the bottleneck that kills you. TensorZero's Rust core makes it the obvious choice for the hot path. The episode-based inference tracking — where multi-step agent workflows are grouped into episodes TensorZero Docs with consistent model variant assignment — is exactly the abstraction needed for coordinating multi-turn agent pipelines. Fork the gateway core as BlackRoad's LLM backbone and extend the ClickHouse observability layer for per-agent analytics. The GitOps-friendly TOML configuration means prompt management can live in version control. TensorZero Docs

Mastra — TypeScript-native agent framework from the Gatsby team

mastra-ai/mastra · ~20,600 ⭐ · TypeScript

The strongest candidate for browser-native agent infrastructure in TypeScript. Built by the team behind Gatsby.js (YC-backed), Mastra provides agents, durable workflows with suspend/resume, RAG pipelines, memory, tool registries with Zod schema validation, evals, and MCP server support. Deploys to Vercel, Cloudflare, Netlify, or any Node environment. Y Combinator

The **suspend/resume workflow engine** is the killer feature for BlackRoad. Agents that can checkpoint state, survive disconnections, and resume where they left off GitHub is non-negotiable when running in browsers. The `createTool` API with Zod validation provides exactly the capability registry pattern needed — typed, validated, discoverable tools that agents can reason about. GitHub Fork as the core TypeScript agent runtime; the Gatsby team's experience scaling build infrastructure to tens of thousands of concurrent nodes validates they understand the scaling challenge.

Mem0 — the memory layer 30K agents actually need

mem0ai/mem0 · ~41,000 ⭐ · Python, TypeScript SDK

An intelligent memory system GitHub using a hybrid architecture — vector DB, key-value store, and graph DB GitHub — with an AUDN cycle (Add/Update/Delete/No-op) where the LLM itself decides how to manage memories. VirtusLab Achieves **26% better accuracy than OpenAI Memory** with 91% lower latency and 90% fewer tokens versus full-context approaches. GitHub arXiv AWS selected Mem0 as the exclusive memory provider for their Agent SDK. AWS Morningstar

With 30K agents and 1M+ users, memory management is an existential concern. Mem0's multi-level scoping (user, session, agent) prevents memory from becoming an undifferentiated blob. GitHub The graph memory component enables agent-to-agent knowledge sharing — Agent A's discoveries become retrievable by Agent B without re-querying. The JS SDK makes browser-adjacent integration straightforward. The memory scoring layer (relevance × importance × recency) is the right approach for intelligent context management within browser resource constraints. GitHub

Rig — the Rust agent framework that compiles to WASM

0xPlaygrounds/rig · ~4,000 ⭐ · Rust

A modular Rust library for LLM-powered agents with trait-based provider abstraction, RAG with pluggable vector stores, prompt chaining, and multi-provider support. GitHub Used by Dria (decentralized AI), Nethermind, and Neon. GitHub The **state machine example**

repo (rig-agent-state-machine) demonstrates event-driven agent lifecycle management. GitHub

Rig is the most mature pure-Rust agent framework available, and the WASM compilation story is its secret weapon. Compile agent logic to WebAssembly for browser-side execution with zero-cost Rust abstractions. The modular crate structure (rig-core plus provider and vector store crates) lets you cherry-pick only what's needed. The agent-as-builder pattern and trait-based extensibility align with Rust idioms BlackRoad's team already uses. Fork rig-core as the Rust backbone; compile to WASM for lightweight browser-side agent runtimes.

Solace Agent Mesh — the event-driven architecture BlackRoad is already designing

SolaceLabs/solace-agent-mesh · ~1,000 ⭐ · Python

A fully asynchronous, event-driven multi-agent architecture where agents communicate via an event mesh (pub/sub), an orchestrator automatically decomposes tasks and delegates to specialists, and agents discover each other via the A2A protocol. GitHub No direct calls between agents — everything flows through events.

This is the **most architecturally aligned** project in the entire list. BlackRoad's event bus coordination model is essentially what Solace Agent Mesh implements: decoupled agents, event-driven communication, automatic task decomposition, and agent discovery. Study the A2A protocol implementation as a ready-made agent-to-agent standard. The orchestrator pattern (automatic task breakdown and delegation) is directly portable. The gap: it's Python-only, so the patterns need porting to TypeScript/Rust for browser execution.

VoltAgent — observability-first TypeScript agents

VoltAgent/voltagent · ~6,900 ⭐ · TypeScript

TypeScript agent framework with **resumable streaming** (clients reconnect to in-flight streams after browser refresh), supervisor/sub-agent coordination, OpenTelemetry-based tracing, and a declarative workflow engine. GitHub Integrates with Hono for lightweight edge deployment.

The resumable streaming capability alone makes VoltAgent worth studying — browser agents disconnect and reconnect constantly, and maintaining stream continuity across those interruptions is a hard problem VoltAgent has already solved. The supervisor-agent hierarchy provides a natural coordination model: a supervisor agent manages a fleet of worker agents, handling failures and rebalancing. GitHub The tool lifecycle hooks (pre/post execution with cancellation) provide the instrumentation points needed for audit logging.

Three more worth your attention

letta-ai/letta (~13K ⭐, Python/TS) pioneered the "LLM as Operating System" concept with the MemGPT paper. Its **.af (Agent File) format** — a portable container for serializing agents — is essentially "Docker for AI agents" and deserves adoption as an industry standard. GitHub GitHub The two-tier memory system (core blocks in context + archival search) and shared memory blocks for multi-agent coordination are directly applicable.

conductor-oss/conductor (~18K ⭐, Java with TS/Go SDKs) is Netflix's battle-tested workflow engine, now supporting AI agent workflows. Its durable execution model (workflows survive crashes, can be replayed) GitHub and worker-based pull model (workers request tasks from a queue) are proven at billions of executions.

ax-llm/ax (~2,500 ⭐, TypeScript) brings DSPy's declarative approach to TypeScript — define type signatures like `"review:string -> sentiment:class"` and the framework handles prompt engineering automatically. At 30K agents, manual prompt tuning is impossible; Ax's automatic prompt optimization (MiPRO, ACE, GEPA) lets agent fleets self-improve. Its AxJSRuntime executes JavaScript in a persistent session — directly relevant to browser-native execution.

Layer 2: Browser-native runtimes, WASM, and OS primitives

This is the substrate — the actual execution environment where agents live. The architectural decision that matters most here is **how to sandbox 30,000 agents in a browser process** without V8 melting down. The answer involves WASM, lightweight JS engines, and component-model composition.

Cloudflare workerd — the nanoservices pattern for 30K agents

cloudflare/workerd · ~6,500 ⭐ · C++

The open-source runtime behind Cloudflare Workers, Cloudflare implementing "nanoservices" — multiple isolated Workers in the same process GitHub using **V8 isolates as security boundaries** instead of separate processes. Same-thread function-call performance between agents, with each isolate having its own memory sandbox. GitHub

This is the **single most important architectural pattern** for BlackRoad. Instead of 30K processes (impossible) or 30K Web Workers (nearly impossible), agents run as V8 isolates within shared processes — each sandboxed but with zero context-switch overhead. The capability-based binding model (agents can only access explicitly granted resources) maps perfectly to zero-trust agent governance. GitHub Fork workerd and replace Cloudflare's orchestration with BlackRoad's mesh network. Implement Durable Objects as persistent agent state. GitHub Study the compatibility date system for backwards-compatible API

evolution. GitHub

rquickjs — 210KB JavaScript engines, 30K of them

DelSkayn/rquickjs · ~1,400 ⭐ (1.4M+ crate downloads) · Rust

High-level Rust bindings for QuickJS-NG, the tiny (**210 KiB**) embeddable JS engine supporting full ES2023. GitHub A complete runtime lifecycle completes in under **300 microseconds**. GitHub Custom module loaders, GC-integrated Rust objects, serde serialization.

Do the math: V8 isolates cost ~10MB+ each. At 30K agents, that's 300GB. QuickJS at 210KB each? That's **6.3GB** — achievable. Sub-300µs instantiation means dynamic agent spawning is essentially free. Each agent gets its own JavaScript interpreter with its own heap — no shared-state concurrency bugs. It compiles to WASM, so agents can run JS interpreters inside WASM sandboxes in the browser. This is the hidden gem that makes the "30K agents in a browser" number physically possible. Define agent behaviors as JS modules loaded via custom `ModuleLoader`.

Wasmtime and Wasmer — the WASM runtime duopoly

bytecodealliance/wasmtime · ~16,200 ⭐ · Rust wasmerio/wasmer · ~20,400 ⭐ · Rust

Wasmtime (Bytecode Alliance: Mozilla, Fastly, Intel, Microsoft) provides GitHub the `PoolingAllocatorConfig` for pre-allocating memory pools across thousands of concurrent WASM instances and **epoch-based interruption** for fine-grained agent scheduling. Wasmer uniquely compiles to run inside browsers via its `js` Cargo feature Rust and offers the Singlepass compiler for sub-millisecond compilation — ideal for dynamically loading agent plugins. GitHub

Use both strategically: Wasmtime server-side (pooling allocator for batch agent execution, Component Model for typed interfaces) and Wasmer browser-side (the `js` feature compiles the runtime to WASM that runs in the browser's JS engine). The Component Model support in Wasmtime enables defining agent interfaces in WIT (WebAssembly Interface Types) — typed contracts between agents regardless of implementation language.

Extism — the universal plugin system for agent extensibility

extism/extism · ~4,900 ⭐ · Rust core, 15+ language SDKs

A universal plug-in system built on WASM. Users write plugins in any language (Rust, JS, Go, Python), compile to WASM, and run them with built-in **runtime limiters, host-controlled HTTP, and persistent module-scope variables**. The JS SDK works in browsers, Node, Deno, Bun, and Cloudflare Workers. GitHub

Extism solves the agent extensibility problem cleanly. Users write custom agent behaviors in their preferred language, compile to WASM, and BlackRoad loads them safely. The runtime limiters enforce deterministic execution; host-controlled HTTP means agents can't make unauthorized network calls. The "kernel" WASM module that manages plugin memory is a particularly elegant pattern — Extism essentially implements a tiny OS kernel in WASM for memory management.

jco — the JavaScript-to-Component-Model bridge

bytecodealliance/jco · ~880 ⭐ · JavaScript, Rust

Provides a complete JavaScript-native toolchain for the WASM Component Model: build components from JS/TS (via ComponentizeJS embedding SpiderMonkey), transpile WASM components into ES modules for browsers, GitHub and includes WASI Preview2 shims with **configurable sandboxing** (`enableNetwork: false` , custom filesystem preopens). GitHub

jco is the bridge that makes the Component Model usable in browsers. Define agent interfaces in WIT (`interface agent { handle-message: func(msg: message) -> result; }`), implement in any language, and jco transpiles them into standard ES modules that run natively in the browser. GitHub The sandboxing configuration maps directly to zero-trust agent permissions. GitHub This is how you get cross-language agent interoperability in a browser context.

Three essential OS primitives for the browser

nicbarker/clay — while not in the subagent results, worth noting that **nicbarker/clay** provides a microsecond-fast UI layout engine in C that compiles to WASM for browser rendering.

sql-js/sql.js (~13K ⭐, JS/C) gives every agent its own **SQLite database running entirely in WASM** inside the browser. Combined with OPFS, this provides persistent, queryable agent state without any server. Use Web Workers for concurrent database access across agents. GitHub

hughfenghen/opfs-tools (~700 ⭐, TypeScript) wraps the browser's Origin Private File System with a developer-friendly API — the only browser API providing true file system semantics with gigabytes of persistent, sandboxed storage per origin. This becomes BlackRoad OS's native filesystem layer.

RingsNetwork/rings (~300 ⭐, Rust) is a fully decentralized P2P network built on WebRTC and Chord DHT that compiles to both native and browser WASM from the same codebase. The Chord DHT provides **O(log N) routing** for 30K agents without a central coordinator — the same Rust code runs natively on servers and as WASM in browsers. GitHub

Layer 3: OS and kernel patterns that translate to browser agents

The conceptual leap: traditional OS primitives — process scheduling, IPC, capability security, supervision trees — map directly to agent OS primitives. The repos below aren't meant to be forked wholesale; they're **architectural blueprints** for building the agent equivalent of an operating system.

Lunatic — the closest analog to what BlackRoad needs

lunatic-solutions/lunatic · ~4,800 ⭐ · Rust

An Erlang-inspired runtime GitHub that spawns super-lightweight WASM "processes" with Erlang-style supervision, mailbox-based message passing, **per-process capability-based permissions**, and preemptive scheduling GitHub via fuel metering. Supports distributed nodes and hot code reloading.

Lunatic is what you'd get if you designed an OS specifically for running thousands of isolated agents. Each WASM process has its own stack, heap, and syscalls. The permission model (per-process access to filesystem, memory, network) translates directly to per-agent permissions. Supervision trees handle fault recovery automatically — when an agent crashes, its supervisor restarts it with clean state. The distributed node support provides a template for multi-server agent orchestration. Study this before anything else in this category.

Nebulet — the WASM microkernel thought experiment

nebulet/nebulet · ~2,300 ⭐ (archived) · Rust

A proof-of-concept microkernel that runs WebAssembly modules in Ring 0 within a **single address space**. Uses Cranelift to JIT-compile WASM to native code. Eliminates context-switch overhead by treating syscalls as function calls.

Archived but architecturally seminal. Nebulet proves the concept BlackRoad needs: running sandboxed code (WASM) in a single-address-space kernel with near-zero IPC cost between agents. Language-level sandboxing (WASM's memory safety) replaces hardware MMU isolation — exactly the model for a browser context where hardware isolation isn't available. The tasks-communicate-through-channels pattern is the right primitive for agent-to-agent messaging.

Ractor — Erlang's gen_server in Rust, used at Meta

slawlor/ractor · ~1,600 ⭐ · Rust

A pure-Rust actor framework closely modeling Erlang's gen_server, GitHub featuring **four-priority message channels** (signal > stop > supervision > regular messages), supervision trees, and distributed cluster support. Meta uses it in production for Rust thrift overload protection. Featured at RustConf'24.

The four-priority channel design is the key insight: not all agent messages are equal. System signals (shutdown, rebalance) must preempt regular work. Supervision events (agent crashed, agent started) need their own priority lane. Regular agent-to-agent messages flow at normal priority. This maps perfectly to agent OS message scheduling. The distributed cluster module (ractor_cluster) provides EPMD-like discovery for multi-server agent distribution. Fork the supervision semantics for agent fault recovery and the priority channels for agent message scheduling.

Tokio and Crossbeam — the concurrency foundations

tokio-rs/tokio (~28K ⭐, Rust) is the gold standard for async scheduling. Its **work-stealing scheduler** with fairness guarantees (no starvation, bounded delay) is the reference implementation for how to schedule 30K concurrent agents. The channel primitives (mpsc, broadcast, watch, oneshot) map directly to agent messaging patterns. Study the waker-based notification system for building agent event loops.

crossbeam-rs/crossbeam (~7,500 ⭐, Rust) provides the low-level building blocks underneath: MPMC channels Rustasync with Go-style `select!`, work-stealing deques Rustasync (the data structure behind Tokio's scheduler), and epoch-based concurrent memory reclamation. Rustasync The bounded channels Rustasync with backpressure prevent any single agent from overwhelming the system — critical for preventing runaway agents from destabilizing the OS.

seL4 and Tock — capability-based security models

seL4/seL4 (~5K ⭐, C/Haskell) is the formally verified microkernel where **every operation requires an unforgeable capability token**. Port the CSpace model (capability spaces) to a browser context: agents hold cryptographic tokens determining which resources, other agents, and APIs they can access. The CapDL (Capability Distribution Language) provides a declarative way to specify agent permission topologies.

tock/tock (~5,500 ⭐, Rust) runs mutually distrustful applications isolated by **Rust's type system rather than hardware**. The capsule architecture — where untrusted extensions are sandboxed by the type system alone — demonstrates how to achieve isolation purely through language-level mechanisms in a browser. The grant mechanism (per-process kernel memory allocation) maps to per-agent resource budgets.

Two more architectural references

redox-os/kernel (~15K ⭐, Rust) provides the "everything is a URL/scheme" IPC model — agent services become URL-addressable resources, enabling natural service discovery. **asterinas/asterinas** (~8K ⭐, Rust) pioneers the "framekernel" pattern: confine unsafe code to a small, auditable framework (OSTD, GitHub ~15K LoC, **14% of the codebase**) while the rest runs in safe Rust. Asterinas This TCB ratio is the target for a browser-native agent OS. OSTD is published as a standalone crate and provides reusable safe abstractions for tasks, scheduling, and memory management. Asterinas

---

Layer 4: Dev tooling, CLIs, and build systems

The tooling layer determines developer velocity. With a polyglot stack (Rust + TypeScript + Python) across many repos, standardization and speed are everything.

Nushell — structured data pipelines for agent fleet management

nushell/nushell · ~38,800 ⭐ · Rust

A modern shell treating all I/O as structured data (tables, records, lists) Rost Glukhov with native JSON, YAML, CSV, and TOML support. Strongly typed with excellent error messages. GitHub Extensible via Rust plugins.

Replace bash scripting across the fleet with type-safe, structured commands. Query agent fleet status, parse audit ledger entries, manage node configurations — all with structured data pipelines that prevent the silent failures plaguing traditional shell scripts. Create custom Nushell plugins (`nu_plugin_mesh` for topology management, `nu_plugin_agents` for fleet queries). The built-in SQLite integration and Polars DataFrame support enable ad-hoc analytics directly in the shell.

Moon — the polyglot monorepo orchestrator

moonrepo/moon · ~3,700 ⭐ · Rust

A Rust-built monorepo tool positioned between Bazel (too complex) and just/make (too simple). Moonrepo Smart hashing, remote caching, integrated toolchain, project graph generation, task inheritance, and **WASM-based extensibility**. Supports JavaScript, TypeScript, Rust, Go, and Python.

Moon's task inheritance means defining `build`, `test`, `lint` once at the root level with every project inheriting automatically. GitHub The WASM plugin system lets you write custom BlackRoad extensions (edge deployment, agent fleet testing) that run

deterministically. GitHub The project graph enables CI optimization — only rebuild packages affected by a change. Moonrepo Pair with **vercel/turbo** (~26K ⭐, Rust) for the TypeScript-specific workspace, where Turborepo's content-aware caching and `--affected` flag shine. Rost Glukhov

mise-en-place — one tool for all versions and tasks

jdx/mise · ~12,000 ⭐ · Rust

Replaces asdf, nvm, pyenv, rbenv, direnv, and Makefiles in one tool. Manages Node.js, Python, Rust, Go, and hundreds more with per-project `.mise.toml` configuration. Full re-implementation (not a wrapper) of the asdf plugin ecosystem.

A developer switching between the agent framework (Rust), dashboard (TypeScript), and ML pipeline (Python) needs exact version consistency. One `mise.toml` per repo handles this. The task runner orchestrates cross-language builds (`mise run build:agents` triggers Cargo + npm + pytest in order). No shims — real PATH manipulation for better debugging. Paranoid security mode for CI environments.

Ratatui and Bubble Tea — terminal dashboards for agent fleets

ratatui/ratatui (~19,100 ⭐, Rust) provides sub-millisecond TUI rendering for the Rust side — build `blackroad-monitor` showing live agent metrics, mesh topology, and audit trails with minimal resource overhead on edge nodes. The immediate-mode rendering paradigm (no retained widget state) maps to reactive agent state.

charmbracelet/bubbletea (~30K ⭐, Go) brings the Elm Architecture to terminal UIs. The broader Charm ecosystem includes `wish` for SSH-accessible TUI apps GitHub — SSH into any edge node and get a live agent dashboard. The message-passing state management model (Init/Update/View) mirrors actor model patterns. GitHub

The build and CLI foundation

casey/just (~30K ⭐, Rust) is the command runner every repo needs — dead simple, readable, parameterized recipes (`just deploy --node=pi-12`), no arcane Makefile syntax. Public domain (CC0).

biomejs/biome (~16K ⭐, Rust) replaces ESLint + Prettier with a single Rust binary that's **35x faster than Prettier**. One `biome.json` standardizes all TypeScript repos. Compiles to WASM for browser-based linting.

clap-rs/clap (~14K ⭐, Rust) is the CLI parser standard. Pair with **moonrepo/starbase** (~130 ⭐, Rust) — the hidden gem that provides a full CLI application framework (lifecycle management, diagnostics, tracing, shell argument parsing with piping/redirection) on top of

Clap. Starbase powers Moon and Proto under the hood.

backstage/backstage (~28K ⭐, TypeScript) is the CNCF developer portal standard (Spotify-created). Build a BlackRoad mission control portal: agent fleet status, mesh topology visualization, strategy deployment pipeline, and software templates for scaffolding new agents.

How these pieces compose into an agent OS

The real value emerges in composition. Here is how these repositories map to a coherent architecture:

Agent OS Layer	Primary Repos	Function
LLM Gateway	TensorZero (hot path) + LiteLLM (management)	Sub-ms model routing for 30K agents
Agent Runtime	Mastra (TS) + Rig (Rust/WASM)	Browser-native agent execution
Agent Memory	Mem0 (hybrid store) + Letta (.af format)	Per-agent and shared memory
Event Coordination	Solace Agent Mesh patterns + Ractor (priority channels)	Decoupled, event-driven agent communication
Agent Sandboxing	workerd (V8 isolates) or rquickjs (lightweight JS) + Extism (WASM plugins)	Isolation for 30K concurrent agents
Component Model	jco + wasm_component_layer	Cross-language agent interfaces via WIT
Storage	sql.js + opfs-tools	Persistent browser-native state
P2P Mesh	Rings Network	Decentralized agent-to-agent routing
Scheduling	Tokio patterns + Crossbeam primitives	Work-stealing agent scheduler
Security	seL4 capability model + Tock capsule pattern	Agent permissions and isolation

Build System	Moon + Turborepo + mise	Polyglot monorepo orchestration
Developer Portal	Backstage + Ratatui dashboards	Mission control and observability

The most non-obvious insight from this research: **rquickjs is likely more important than any WASM runtime** for hitting the 30K agent target in a browser. At 210KB per interpreter GitHub versus 10MB+ per V8 isolate, QuickJS makes the memory math work. Pair it with Extism for WASM-sandboxed tool execution when agents need to run untrusted code, and you have a two-tier execution model: lightweight QuickJS for agent logic, WASM sandboxes for agent tools.

Conclusion

The strongest architectural signal across all four layers is **convergence on the actor/event-mesh model**. Lunatic, Ractor, Solace Agent Mesh, workerd's nanoservices, and Nebulet's WASM microkernel all independently arrived at the same pattern: lightweight isolated execution units communicating through typed message passing with supervision trees for fault recovery. This isn't coincidental — it's the natural architecture for managing thousands of concurrent, potentially untrusted execution contexts, which is exactly what an agent OS is.

The three repos that deserve the deepest architectural study are **workerd** (how Cloudflare runs millions of isolates in shared processes), **Lunatic** (Erlang's proven supervision model applied to WASM), and **Solace Agent Mesh** (event-driven agent coordination without coupling). The most underrated repo in the entire list is **rquickjs** — it's the one that makes the numbers physically possible. And the most surprising finding is that **Asterinas's framekernel pattern** — confining all unsafe code to 14% of the codebase while maintaining monolithic performance — provides a concrete, measurable security target for BlackRoad's own kernel layer.